

Bộ câu hỏi & trả lời dự kiến cho hội đồng bảo vệ

Tài liệu tổng hợp các câu hỏi có xác suất cao được hội đồng đặt ra khi bảo vệ khóa luận Dental AI Assistant, kèm trả lời có tham chiếu mã nguồn & tài liệu. Cấu trúc câu trả lời theo mẫu: **Khẳng định ngắn** → **Giải thích** → **Dẫn chứng trong code/docs**.

A. Câu hỏi về kiến trúc tổng thể

A1. Tại sao chọn RAG thay vì fine-tuning LLM?

Trả lời: Với bài toán y khoa, RAG phù hợp hơn vì 3 lý do:

1. **Cập nhật kho tri thức rẻ và nhanh** — chỉ cần thêm bài viết mới và chạy lại `ingest`, không cần training lại mô hình hàng tỷ tham số.
2. **Truy vết nguồn (Citation Traceability)** — mỗi câu trả lời đều đính kèm danh sách tài liệu gốc. Fine-tuning không làm được điều này vì tri thức bị "hòa tan" vào trọng số.
3. **Chống ảo giác có kiểm soát** — ta dùng prompt-based guardrail (Rule 2, Rule 3) buộc LLM chỉ trả lời trong phạm vi context, đây là cơ chế mà fine-tuning không đảm bảo được 100%.

Tham chiếu: `HALLUCINATION_GUARDRAILS.md`, `src/lib/constants.py` — `AI_SYSTEM_INSTRUCTIONS`.

A2. Kiến trúc tổng thể hệ thống gồm những thành phần nào?

Trả lời: Hệ thống có 4 tầng:

1. **Frontend (Next.js 16)** — UI/UX, Zustand state, SSE consumer.
2. **Backend API (FastAPI)** — REST + SSE streaming, JWT/2FA, lifespan warmup.
3. **RAG Pipeline (`src/agent`, `src/retriever`, `src/lib`)** — Query Rewrite → Extract + Expand (song song) → Hybrid Search (FAISS + BM25 + RRF) → LLM Stream.
4. **Data Layer** — MySQL (user + chat history) + FAISS (vector DB) + BM25 corpus (in-memory).

Sơ đồ end-to-end tại `TECHNICAL_FLOW.md` §Sơ đồ tổng hợp.

A3. Vì sao dùng FAISS thay vì Pinecone, Milvus, Weaviate,...?

Trả lời: Với 762 tài liệu, FAISS `IndexFlatIP` cho brute-force similarity là lựa chọn tối ưu vì:

- **Precision tuyệt đối** — không có lỗi xấp xỉ ANN (HNSW, IVF).
- **Độ trễ < 5 ms** cho 762 docs × 1536 dim.
- **Không phụ thuộc dịch vụ ngoài** — FAISS chạy inprocess, không cần mở container riêng.
- **Zero chi phí** — không phải trả tiền storage cloud.

Khi dataset mở rộng > 1 triệu doc, có thể chuyển sang `IndexHNSWFlat` hoặc `IndexIVFPQ` mà không sửa kiến trúc (lớp `Retriever` bọc ngoài).

Tham chiếu: `src/retriever/ingest.py` dòng 125 — `faiss.IndexFlatIP(actual_dim)`.

B. Câu hỏi về Retrieval

B1. Tại sao phải dùng Hybrid Search mà không dùng riêng Vector Search?

Trả lời: Hai kênh bù trừ nhau:

Kênh	Thế mạnh	Điểm yếu
FAISS (Vector)	Hiểu ngữ nghĩa: "đau răng" ≈ "nhức răng"	Có thể miss khi khớp chính xác từ khóa y khoa hiếm
BM25 (Keyword)	Bắt chính xác thuật ngữ + mã bệnh	Không hiểu đồng nghĩa, bị giảm hiệu quả trên ngôn ngữ đa âm tiết

Hybrid qua RRF cho phép doc xuất hiện ở **cả hai kênh** được boost tự nhiên — đây là kỹ thuật state-of-the-art được dùng trong các hệ thống search hiện đại (Elastic, Weaviate, Vespa).

Chi tiết: [RRF_EXPLANATION.md](#) + [BM25_TOKENIZATION.md](#).

B2. Công thức RRF dùng trong dự án khác gì so với công thức gốc?

Trả lời: Công thức gốc (Cormack 2009) không có trọng số:

$$\text{RRF}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

Trong dự án, tôi mở rộng thành **Weighted RRF**:

$$\text{RRF}(d) = \frac{w_v}{k + \text{rank}_{\text{FAISS}}(d) + 1} + \frac{w_b}{k + \text{rank}_{\text{BM25}}(d) + 1}$$

với w_v, w_b được **điều chỉnh động** theo loại câu hỏi:

- Câu chứa "chi phí", "quy trình", "bao nhiêu", "bao lâu" → $w_v = 0.3, w_b = 0.7$ (ưu tiên keyword).
- Các câu khác → $w_v = w_b = 0.5$.

Code: [src/retriever/search.py](#) — `_is_keyword_heavy()` và `_hybrid_score()`.

B3. Tại sao hằng số $k = 60$ trong RRF?

Trả lời: $k = 60$ là giá trị khuyến nghị của bài báo gốc (Cormack 2009), có tác dụng **làm mượt độ dốc điểm**. Nếu k nhỏ (ví dụ 10), top-1 vượt trội quá mức → hybrid gần như tương đương chọn kết quả top-1 của mỗi kênh. Nếu k quá lớn (100+), mọi rank gần như bằng nhau, hybrid chỉ còn đếm "xuất hiện ở mấy danh sách". $k = 60$ là điểm cân bằng được validate qua nhiều benchmark.

Ví dụ tính toán chi tiết tại [RRF_EXPLANATION.md §2](#).

B4. Multi-Query Expansion giúp gì?

Trả lời: Giải quyết **Vocabulary Mismatch** — khi user hỏi "chi phí" nhưng tài liệu viết "bảng giá". LLM sinh thêm 2 biến thể → tổng 3 queries → điểm RRF của mỗi doc được **cộng cross-query** → doc xuất hiện ở nhiều biến thể có điểm cao hơn. Đây là **implicit voting**: mỗi biến thể = 1 lá phiếu.

Cái giá: +1 LLM call (nhANH, max_tokens=200) và × 3 lần Hybrid Search (FAISS 762 docs cost < 5 ms nên không đáng kể).

Chi tiết: [MULTI_QUERY_EXPANSION.md](#).

B5. Vì sao Extract Category và Expand Query chạy song song được?

Trả lời: Hai tác vụ **không phụ thuộc dữ liệu của nhau** — cả hai chỉ cần `rewritten_question` làm input. Tôi dùng `ThreadPoolExecutor(max_workers=2)` để gọi 2 LLM call đồng thời:

```
with ThreadPoolExecutor(max_workers=2) as executor:
    future_extract = executor.submit(_safe_extract)
    future_expand = executor.submit(_safe_expand)
    categories = future_extract.result()
    expanded_queries = future_expand.result()
```

Tổng thời gian = `max(T_extract, T_expand)` thay vì tổng. Với Ollama local, tôi set `OLLAMA_NUM_PARALLEL=3` trong `start_ollama.bat` để đảm bảo không bị tuần tự hóa ở tầng server.

Code: `src/agent/chatbot.py` — `answer_stream()`.

C. Câu hỏi về Embedding

C1. Tại sao hỗ trợ 2 embedding engine?

Trả lời: Có 3 lợi ích:

1. **Linh hoạt chi phí/chất lượng** — local miễn phí (dev), OpenAI chất lượng cao (prod).
2. **So sánh benchmark** — build index cả 2, đánh giá chéo trên cùng bộ câu hỏi.
3. **Dự phòng** — nếu OpenAI sập hoặc hết quota, đổi `.env` sang local là chạy được.

Tôi dùng **Strategy Pattern** (`src/retriever/engines.py`) để chuyển engine tại runtime qua biến môi trường, không cần sửa code.

Tham chiếu: [EMBEDDING_ENGINE.md](#).

C2. Tại sao L2-normalize và dùng `IndexFlatIP`?

Trả lời: Về mặt toán học:

$$\text{CosineSim}(\hat{Q}, \hat{D}) = \hat{Q} \cdot \hat{D} \quad \text{với} \quad |\hat{Q}| = |\hat{D}| = 1$$

Tôi normalize **1 lần lúc ingest** để mọi vector đều có chuẩn L2 = 1, sau đó mỗi truy vấn chỉ cần **Inner Product (nhân-cộng)** thay vì Cosine đầy đủ. Lợi ích:

- **Tăng tốc ~3x** — bỏ 2 SQRT + 1 DIV mỗi cặp.
- **Tận dụng SIMD + FMA** — `VFMADD231PS` xử lý 16 float32 mỗi cycle.
- **Chi phí normalize được khấu hao** (amortized) qua hàng nghìn truy vấn.

Chứng minh hình thức: [COSINE_TO_IP_PROOF.md](#).

C3. Tại sao chọn `keepitreal/vietnamese-sbert`?

Trả lời: Model được huấn luyện trên corpus tiếng Việt, dựa trên PhoBERT:

- Chất lượng tốt cho tiếng Việt so với `multilingual-sbert`.
- 768 dim — đủ biểu diễn ngữ nghĩa, không quá nặng.
- ~420 MB — chấp nhận được với máy sinh viên.
- Load offline qua `local_files_only=True` + `@lru_cache` (singleton).

Chi tiết kỹ thuật: `src/retriever/engines.py` — `LocalEngine` + `_load_sbert_model()`.

D. Câu hỏi về LLM & Prompt Engineering

D1. Giải thích 9 quy tắc của system prompt?

Trả lời: 9 quy tắc định nghĩa hành vi LLM:

Rule	Mục đích
1	Không mở đầu cảm thán, đi thẳng nội dung
2	Strict Grounding — chỉ dùng context, không kiến thức tự có
3	Từ chối cố định khi không có dữ liệu
4	Giới hạn domain nha khoa
5	Ngôn ngữ chuyên nghiệp, dễ hiểu
6	Plain text, không Markdown
7	Liệt kê bằng -, không dòng trống giữa
8	Không tự thêm disclaimer (backend tự thêm)
9	Chống ám thị chi tiết — không trình bày quy trình của 1 thương hiệu như quy trình chung

Rule 2, 3, 9 là **lỗi chống ảo giác**. Rule 6, 7, 8 là format để UI hiển thị ổn định.

Full prompt: `src/lib/constants.py` — `AI_SYSTEM_INSTRUCTIONS`. Giải thích từng rule: `PROMPT_ENGINEERING.md`.

D2. Temperature được set thế nào và vì sao?

Trả lời:

LLM Call	Temperature	Lý do
<code>rewrite_query</code>	0.0 (STRICT)	Rewrite cần deterministic, không sáng tạo
<code>extract_category</code>	0.0 (STRICT)	Phân loại cần chính xác
<code>expand_queries</code>	0.5	Cần đa dạng từ đồng nghĩa, không lặp
<code>answer_stream</code>	0.3 (NORMAL)	Đủ tự nhiên, vẫn bám sát context

Nguyên tắc: nhiệm vụ càng cần **chính xác** → temperature càng **thấp**. Temperature = 0.0 = argmax token, loại bỏ hoàn toàn randomness.

D3. Tại sao không dùng Function Calling / Tool Use thay prompt?

Trả lời: Function Calling phù hợp khi LLM **sinh cấu trúc JSON phức tạp** để trigger API. Trong dự án, tôi cần LLM trả về **chuỗi ngắn đơn giản** (query rewrite, category name) — prompt instruction đủ và rẻ hơn 1 lớp structured output. Hơn nữa, Ollama hỗ trợ function calling không đều, prompt-based đảm bảo tương thích cả cloud/local.

D4. Giải thích cơ chế Rewrite Query?

Trả lời: Rewrite Query luôn chạy trước mọi câu hỏi, kể cả câu đầu tiên, vì 3 mục đích:

- 1. **Chuẩn hóa** — thêm từ khóa y khoa (“tổng quan”, “các loại phổ biến”) giúp retrieval trúng bài khái quát.
- 2. **Ghép ngữ cảnh** — câu follow-up (“có đất không?”) được ghép chủ đề từ câu trước.
- 3. **Chống ám thị chi tiết** — cấm LLM tự thêm tên thương hiệu, vị trí cụ thể (Rule trong `REWRITE_USER_TEMPLATE`).

Format history riêng cho rewrite: 6 message gần nhất, assistant chỉ lấy 120 ký tự đầu để tiết kiệm token.

Code: `src/agent/chatbot.py` — `rewrite_query()` + `format_history_for_rewrite()`.

D5. Vì sao disclaimer do backend tự thêm chứ không để LLM?

Trả lời: 3 lý do:

- 1. **Consistency** — LLM có thể quên hoặc diễn đạt khác nhau qua các lần. Backend append cố định.
- 2. **Không tốn token** — tiết kiệm ~30 tokens output mỗi câu.
- 3. **Compliance** — dễ audit và thay đổi nội dung disclaimer khi cần.

Rule 8 cấm LLM tự thêm để tránh trùng lặp.

E. Câu hỏi về Anti-Hallucination

E1. Hệ thống chống ảo giác thế nào?

Trả lời: 5 tầng phòng thủ (defense-in-depth):

Tầng	Cơ chế	Vị trí
1. Query	Rewrite thêm “tổng quan”, không thêm chi tiết	<code>rewrite_query()</code>
2. Retrieval	Category pre-filter + Overview Boost + Dynamic Weights	<code>search()</code> , <code>_boost_overview()</code>
3. Prompt	9 quy tắc cứng, tập trung Rule 2, 3, 9	<code>AI_SYSTEM_INSTRUCTIONS</code>
4. Temperature	0.0 cho rewrite/extract, 0.3 cho generation	<code>AI_TEMPERATURE</code>
5. Output	Disclaimer cố định + danh sách Sources	<code>answer_stream()</code>

Ma trận phòng thủ chi tiết (loại ảo giác × tầng bảo vệ): [HALLUCINATION_GUARDRAILS.md §5](#).

E2. Detail Suggestion Bias là gì và xử lý thế nào?

Trả lời: Hiện tượng: khi kho dữ liệu chứa nhiều bài về **1 thương hiệu** (Invisalign), LLM có xu hướng trình bày quy trình của thương hiệu đó như quy trình chung của ngành. Vấn đề này xuất hiện mà **không phải là ảo giác truyền thống** (LLM không bịa), mà là **thiên lệch phân phối dữ liệu**.

Giải pháp 3 tầng:

- 1. **Query Rewrite**: thêm “tổng quan” / “các loại phổ biến” để định hướng retrieval vào bài khái quát.
- 2. **Overview Boost**: `_boost_overview()` đẩy bài có section tổng quan lên đầu, trước khi LLM đọc.
- 3. **Prompt Rule 9**: bắt LLM phải ghi rõ “Theo quy trình của [Tên Hãng]...” nếu phải dùng thông tin thương hiệu.

E3. Làm sao verify được câu trả lời đúng?

Trả lời: Có 3 công cụ:

- 1. **Sources panel trên UI** — user click để xem title, section, URL gốc.
- 2. **rewritten_query được log** — giúp debug retrieval: nếu query rewrite sai, cả chuỗi sau bị sai.
- 3. **Log time + rank scores** trong backend (`[TIME-LOG]`) — có thể trace từng LLM call.

Hướng phát triển: tự động highlight câu trong output khớp với câu trong context (citation verification).

F. Câu hỏi về API & Backend

F1. Tại sao chọn FastAPI?

Trả lời:

- **Async native** — `async/await` built-in, phù hợp SSE streaming.
- **Pydantic validation** — tự động validate request body + sinh OpenAPI schema.
- **Dependency Injection** — `Depends(get_chatbot)`, `Depends(get_current_user)` sạch, testable.
- **Hiệu năng** — Starlette + Uvicorn nhanh hơn Flask/Django cho API.

F2. SSE khác gì WebSocket?

Trả lời:

Tiêu chí	SSE	WebSocket
Direction	Server → Client only	Bidirectional
Protocol	HTTP/1.1 hoặc HTTP/2 (chuẩn)	Upgrade từ HTTP
Reconnect	Tự động (built-in EventSource)	Phải tự code
Proxy/firewall	Thân thiện (chỉ là HTTP)	Có thể bị chặn
Use case	Streaming 1 chiều (LLM token)	Chat realtime 2 chiều

Chat của dự án **chỉ cần server đẩy token về** (user không gửi gì giữa chừng) → SSE phù hợp hơn và đơn giản hơn nhiều. Riêng dự án dùng `fetch` + `ReadableStream` thay vì `EventSource` vì cần header `Authorization: Bearer`.

F3. Singleton Chatbot + lifespan có ý nghĩa gì?

Trả lời:

- **Singleton:** `DentalChatbot` load model sbert (~420 MB) + FAISS index + BM25 corpus, không thể load mỗi request.
- **Lifespan:** `api/main.py` gọi `get_chatbot()` trong `asynccontextmanager` → pre-warm ngay khi server start, **request đầu tiên không bị cold-start**. Giảm first-response latency từ ~5 s xuống ~0.5 s.

Code: `api/main.py` + `src/chat/dependencies.py`.

G. Câu hỏi về Bảo mật

G1. Vì sao hash password bằng bcrypt?

Trả lời:

- **Tự động salt** — mỗi hash có salt ngẫu nhiên, cùng password cho ra hash khác nhau → chống rainbow table.
- **Cost factor (rounds)** — mặc định $12 = 2^{12}$ iterations, đủ chậm để chống brute-force.
- **Chuẩn được kiểm chứng** — dùng rộng rãi trong industry, passlib đã wrap sẵn.

Alternative (Argon2) mạnh hơn nhưng bcrypt đủ cho scale này, compatibility cao hơn.

G2. JWT lưu ở đâu và sao không dùng session cookie?

Trả lời:

- JWT lưu ở `localStorage` (key `dental_ai_token`) phía frontend, mỗi request gắn header `Authorization: Bearer`.
- Lý do chọn JWT thay session cookie:
 - **Stateless** — server không cần lưu session, scale ngang dễ.
 - **Frontend/Backend tách biệt** — không vướng Same-Site, CSRF.
 - **Hết hạn tự động** — `exp` claim, verify bằng `python-jose`.

Risk: XSS có thể đánh cắp token. Mitigation: input sanitization phía frontend, không render HTML thô từ user, CSP header nếu deploy production.

G3. 2FA hoạt động thế nào?

Trả lời: Dùng TOTP (Time-based One-Time Password):

1. **Setup:** server sinh secret base32 bằng `pyotp.random_base32()`, tạo QR code (SVG → base64), user quét bằng Google Authenticator.
2. **Verify setup:** user nhập mã 6 số, server verify bằng `pyotp.TOTP.verify(code, valid_window=1)` (chấp nhận ± 30 s trôi đồng hồ).

3. **Login:** sau khi verify password, nếu `is_2fa_enabled=True` → server trả `temp_token` (JWT với `purpose=2fa_verify`, sống 5 phút) thay vì access token thật. Frontend hỏi mã TOTP → gọi `/2fa/login-verify` với `temp_token` + code → nếu đúng, trả JWT thật.

`temp_token` đóng vai trò **proof of password** — không cần lưu trạng thái intermediate trên server.

Chi tiết: [AUTH_SECURITY.md §4](#).

G4. Bạn lưu password bị lộ thì sao?

Trả lời: Password được hash bằng bcrypt (salt riêng mỗi user, 12 rounds). Kể cả database bị lộ, attacker phải brute-force từng user riêng → với password ≥ 8 ký tự ngẫu nhiên, chi phí không khả thi. Ngoài ra, SECRET_KEY (JWT signing) lưu trong `.env`, không commit vào git (`.gitignore`).

H. Câu hỏi về Dữ liệu

H1. Dữ liệu lấy từ đâu? Có vi phạm bản quyền không?

Trả lời:

- Crawl từ các trang y khoa công khai: Vinmec, Pharmacity, VnExpress Sức khỏe.
- Dùng cho mục đích **học thuật / nghiên cứu**, không thương mại.
- Luôn **trích dẫn nguồn** (title + URL) trong câu trả lời → tôn trọng tác giả gốc.
- Nếu thương mại hóa, cần xin license từ các bên này.

H2. Tại sao nâng cấp dataset v1 → v2?

Trả lời: v2 thêm 2 trường:

- **summary** (tóm tắt ≤ 30 từ) — đóng vai trò “anchor ngữ nghĩa” cho embedding, cải thiện retrieval trên bài content dài.
- **content_md** (markdown) — heading, bold cho thuật ngữ quan trọng → embedding bắt được entity y khoa tốt hơn, BM25 cũng tăng TF-IDF cho từ bold.

Quy trình: `tools/upgrade_dataset.py` gọi `gpt-4o-mini` với JSON mode, checkpoint mỗi 50 bài, retry exponential backoff.

Chi tiết: [DATA_SCHEMA.md](#).

H3. Có bao nhiêu bài? Có đủ để cover mọi câu hỏi không?

Trả lời:

- 762 bài, trải đều trên các chủ đề chính: Sâu răng, Niềng răng, Implant, Răng sứ, Nhổ răng khôn, Viêm nha chu,...
- Không đủ cover 100% mọi câu hỏi — và đó là lý do tại sao **Rule 3** tồn tại: khi dữ liệu không có, hệ thống trả lời đúng 1 câu từ chối cố định thay vì bịa.
- Dataset có thể mở rộng dễ dàng: thêm JSON + chạy `ingest` → FAISS được rebuild.

I. Câu hỏi “ác ý” / đào sâu

I1. Nếu cùng một câu hỏi hỏi 2 lần, đáp án có khác nhau không?

Trả lời: Có thể khác ở câu văn nhưng **không khác về thông tin cốt lõi**, vì:

- Temperature generation = 0.3 → có randomness nhỏ.
- Rewrite và Extract = 0.0 → deterministic, retrieval cho ra cùng danh sách tài liệu.
- `expand_queries` = 0.5 → variants có thể khác chút, nhưng RRF cross-query merge làm kết quả vẫn hội tụ.

Có thể set temperature generation = 0.0 nếu cần reproducibility tuyệt đối, đánh đổi bằng văn phong máy móc hơn.

I2. Nếu tắt mạng OpenAI, hệ thống còn chạy không?

Trả lời: Có, nếu:

1. `EMBEDDING_ENGINE=local` (dùng vietnamese-sbert).
2. `LLM_ENGINE=local` (dùng Ollama + qwen2.5).
3. FAISS index `local` đã được build trước đó.

Chỉ cần MySQL và Ollama bật — hệ thống hoạt động hoàn toàn offline.

I3. Latency pipeline khoảng bao nhiêu?

Trả lời: Benchmark điển hình trên máy dev (RTX 3060 6 GB):

Giai đoạn	Cloud (GPT-4.1-mini)	Local (Qwen2.5 1.5B Q4)
Rewrite Query	~0.6 s	~0.4 s
Extract + Expand (song song)	~0.8 s	~0.6 s
Retrieval (3 queries × FAISS+BM25)	~0.05 s	~0.05 s
LLM First Token	~0.4 s	~0.8 s
LLM Generation (250 tokens)	~2.5 s	~4–6 s
Tổng	~4 s	~6–8 s

Log chi tiết hiển thị trong console backend với prefix `[TIME-LOG]`.

I4. Nếu LLM nhận context dài quá 8K tokens thì sao?

Trả lời: Hiện tại Top-K = 10 docs × ~400 tokens = ~4K, dưới ngưỡng an toàn. Nếu dataset mở rộng lớn hơn và context vượt limit:

- GPT-4.1-mini: context window 128K → vẫn an toàn.
- Qwen2.5-1.5B: context 32K → vẫn an toàn.
- Giải pháp mở rộng: thêm **re-ranking** với cross-encoder để nén về Top-5 chất lượng cao, hoặc tóm tắt context bằng LLM thứ 2.

I5. Vì sao không dùng langchain/llamaindex?

Trả lời: Quyết định có chủ ý:

- **Minh bạch** — tự implement từng bước để hiểu sâu thuật toán (RRF, BM25, RAG pipeline), đúng tinh thần khóa luận.
- **Không lock-in** — không phụ thuộc abstraction của framework, dễ debug và customize (ví dụ Overview Boost, Multi-Query Expansion đều là tùy biến không có sẵn).
- **Kích thước bundle nhẹ** — langchain import kéo theo hàng trăm dependencies.
- **Học thuật** — hội đồng thường đánh giá cao code tự viết hơn là wrap thư viện.

I6. Hệ thống có khả năng scale không?

Trả lời:

Trục scale	Giải pháp
Số user đồng thời	Chạy nhiều worker Uvicorn (<code>--workers 4</code>); FastAPI async handles concurrency
Số tài liệu	Thay <code>IndexFlatIP</code> bằng <code>IndexHNSWFlat</code> ; chuyển BM25 qua Elasticsearch
Throughput LLM	Cache kết quả rewrite/extract phổ biến (Redis); batching request
Geographic	Deploy backend gần user (edge) + CDN cho frontend Vercel

Kiến trúc hiện tại là **prototype production-ready cho < 100 user đồng thời**; scale-out cần thêm message queue + distributed vector DB.

J. Câu hỏi về Hướng phát triển

J1. Định hướng mở rộng tương lai?

Trả lời:

1. **Output Validator** — thêm LLM call thứ 5 kiểm tra “câu trả lời có bám sát context không”, gạch đầu dòng nào không có nguồn → mark warning.
2. **Fine-tune embedding trên corpus nha khoa** — retrain sbert với hard negatives từ dataset để tăng precision.
3. **Citation highlighting** — span-level matching giữa answer và context.
4. **Multilingual** — mở rộng dataset tiếng Anh, tiếng Hàn cho nha khoa.
5. **Mobile app** — React Native reuse Zustand store.
6. **Voice interface** — thêm STT/TTS để dùng trên mobile.
7. **Image input** — user upload ảnh răng, integrate CV model.

J2. Cải thiện gì nếu làm lại?

Trả lời: (ăn điểm khiêm tốn)

- **Test coverage** — dự án chưa có unit test; nên thêm pytest cho retriever và auth flow.
- **CI/CD** — GitHub Actions lint + test tự động.
- **Observability** — tích hợp Sentry cho backend error, PostHog cho FE analytics.
- **Semantic cache** — cache embedding của query phổ biến (Redis) để giảm LLM call.
- **Evaluation pipeline** — metrics Recall@K, MRR, Precision@K trên testset có annotation.

K. Checklist thuộc lòng lúc bảo vệ

- ☐ Biết giải thích `.env` từng biến trong 1 câu.
- ☐ Vẽ được sơ đồ 5 bước pipeline (Rewrite → Extract/Expand song song → Retrieval → Context → Stream).
- ☐ Thuộc công thức RRF và biết tại sao $k = 60$.
- ☐ Biết chứng minh $\text{Cosine} \equiv \text{Inner Product}$ khi normalize.
- ☐ Biết tên 9 rule + tối thiểu 3 rule cốt lõi (2, 3, 9).
- ☐ Biết 5 tầng guardrail và 4 loại ảo giác.
- ☐ Biết cách bật/tắt hệ thống (MySQL → Ollama → Backend → Frontend).
- ☐ Biết 2 ca demo ấn tượng: (a) câu hỏi follow-up “có đấy không?”, (b) câu hỏi ngoài domain “công thức nấu phở”.
- ☐ Nắm được benchmark latency cloud vs local.
- ☐ Có sẵn file `.env.example` + `requirements.txt` để giải thích nếu hội đồng hỏi.